

Звіт з лабораторної роботи

1. Тема та мета

Тема: Code Review та рефакторинг коду.

Мета: навчитись проводити взаємне рецензування коду, виявляти проблеми якості, документувати їх та виконувати рефакторинг з підтвердженням регресійними тестами.

Виконала: Євсович Анна, група ПЗПІ-25-1

Репозиторій: <https://github.com/AnnaYevsovych/CratlogGroup4>

2. Результати Code Review

№	Файл / Рядок	Проблема	Категорія	Рекомендація
1	Work.java, р.11	Поле workId не final — може бути змінено після створення	Mutable field	Додати final: private final int workId
2	Creator.java, р.110	hashPassword використовує hashCode() — не криптографічна функція	Security smell	Замінити на SHA-256 або BCrypt
3	Creator.java, р.15	Поле email змінюване, але метод changeEmail відсутній	Design smell	Додати метод changeEmail(String) з валідацією
4	Portfolio.java, р.97	getTags() створює новий ArrayList при кожній ітерації searchByTag	Performance smell	Використати Set<String> для O(1) пошуку
5	WorkTest.java	Метод getDetails() не покритий жодним тестом	Coverage gap	Додати TC-W07 для перевірки формату getDetails()

3. Посилання на PR

PR з коментарями Code Review: <https://github.com/AnnaYevsovych/CratlogGroup4/pull/4>

4. Результати рефакторингу

Рефакторинг 1: Додано final до поля workId (Work.java)

БУЛО:

```
private int workId;
```

СТАЛО:

```
private final int workId;
```

ЧОМУ: Ідентифікатор роботи не повинен змінюватись після створення об'єкта. `final` гарантує immutability.

Рефакторинг 2: Замінено `String.matches()` на `Pattern.compile()` (Creator.java)

БУЛО:

```
email.trim().matches(EMAIL_REGEX)
```

СТАЛО:

```
EMAIL_PATTERN.matcher(email.trim()).matches() // + додано: private
static final Pattern EMAIL_PATTERN = Pattern.compile(EMAIL_REGEX);
```

ЧОМУ: `String.matches()` компілює regex при кожному виклику. `Pattern.compile()` робить це один раз — ефективніше.

Рефакторинг 3: Додано валідацію username (Creator.java)

БУЛО:

```
this.username = username;
```

СТАЛО:

```
if (username == null || username.isBlank()) {      throw new
IllegalArgumentException("Username не може бути порожнім"); }
this.username = username;
```

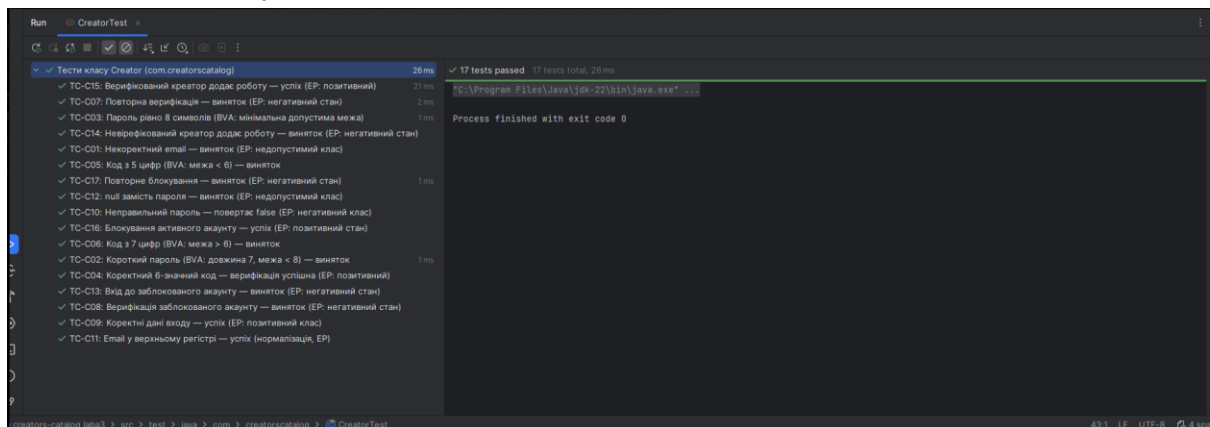
ЧОМУ: Конструктор валідував email і пароль, але не username. Додана перевірка усуває можливий `NullPointerException`.

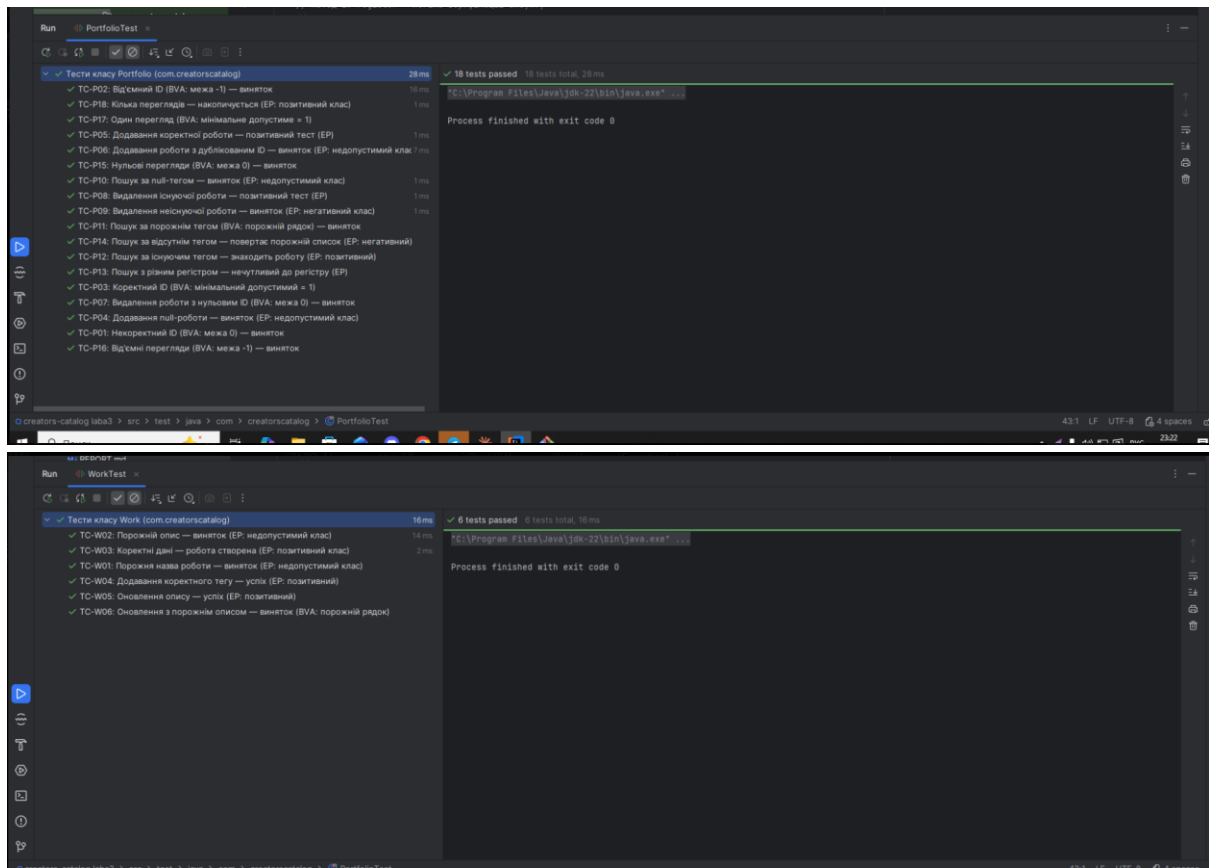
5. Регресійне тестування

Після рефакторингу запущено всі JUnit-тести:

- CreatorTest: 17 tests passed
- PortfolioTest: 18 tests passed
- WorkTest: 6 tests passed

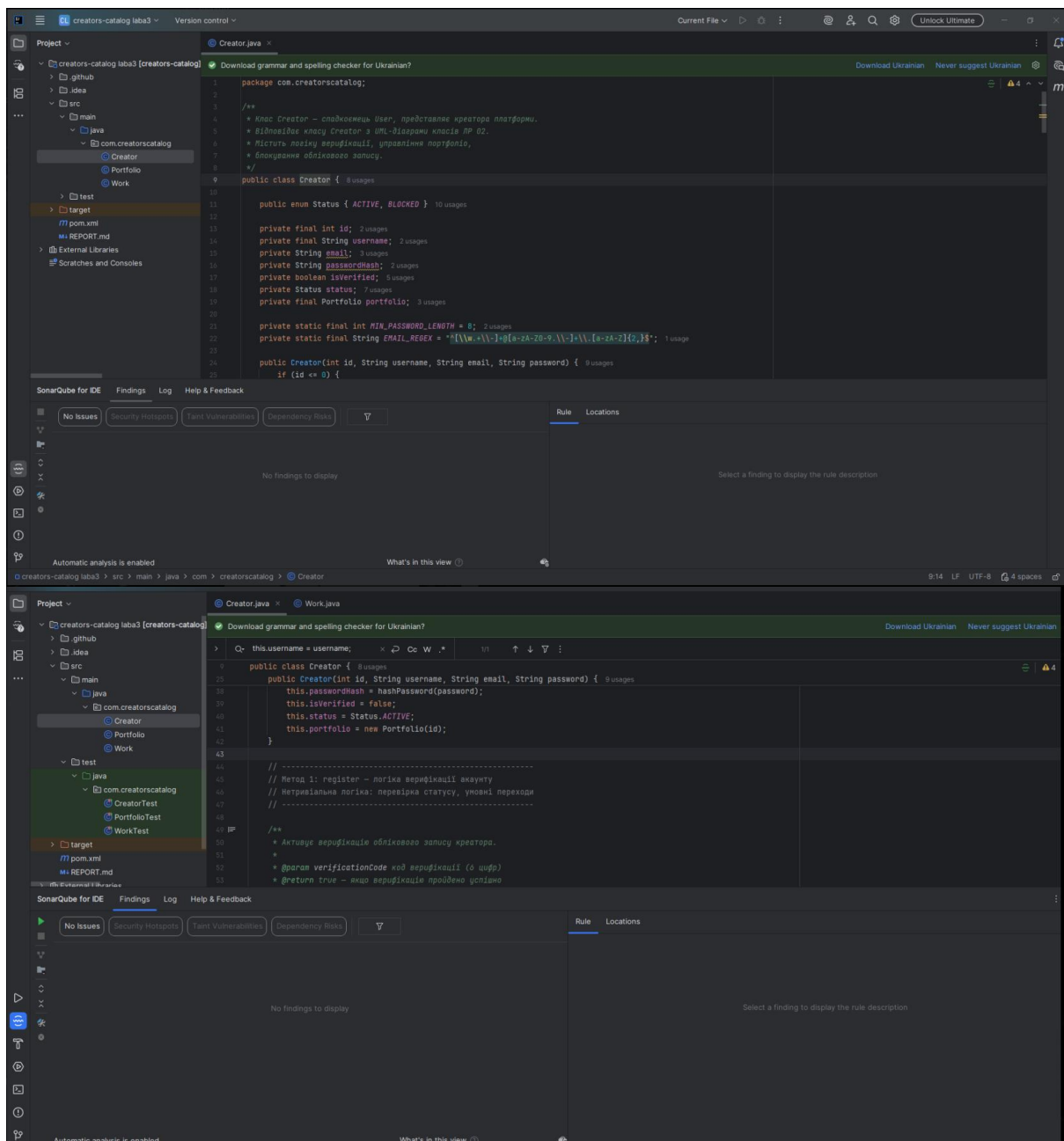
Всього: 41 тест — усі GREEN. Process finished with exit code 0.





6. Порівняння метрик ДО / ПІСЛЯ

Метрика	ДО	ПІСЛЯ
SonarLint issues	0	0
Кількість тестів	41	41
Всі тести green	Так	Так
Валідація username	Відсутня	Додана
Regex компіляція	При кожному виклику	Один раз (Pattern.compile)
Поле workId	Mutable (int)	Immutable (final int)



7. Підсумкова рефлексія

Під час Code Review я навчилась системно аналізувати чужий код і формулювати конкретні рекомендації з обґрунтуванням. Найціннішим досвідом стало усвідомлення різниці між автоматичним аналізом (SonarLint) та ручним рецензуванням — інструменти не помітили проблем, які я знайшла вручну. Рефакторинг навчив мене думати про наслідки змін для тестів: регресійне тестування показало що всі 41 тест залишились green. Надалі буду приділяти більше уваги валідації вхідних даних та immutability полів з самого початку проектування.